# Common Web Vulnerabilities

## 1. Objective

The objective of this task was to learn how to identify and test common web vulnerabilities, including SQL Injection, Cross-Site Scripting (XSS), authentication flaws, IDOR, access-control issues, path traversal, file-upload vulnerabilities, and other OWASP Top 10 vulnerabilities.

---

## 2. Concept/Understanding

| Vulnerability | Concept and understanding |
|---|---|
| **SQL Injection** | Occurs when user input is directly concatenated into SQL queries. Attackers can manipulate queries to access, modify, or delete data. |
| **Cross-Site Scripting (XSS)** | Occurs when user input is reflected or stored without encoding. JavaScript executes in the victim's browser, allowing session theft, defacement, and other attacks. |
| **Authentication Vulnerabilities** | Weak login mechanisms, username enumeration, weak passwords, and session issues can allow attackers to gain unauthorized access. |
| **IDOR (Insecure Direct Object Reference)** | Occurs when applications expose internal object references, such as IDs, without proper access control. Attackers can access other users' data by changing ID values. |
| **Broken Access Control** | Occurs when access-control checks are missing or flawed, allowing users to access resources they should not be able to access. |
| **Path Traversal** | Occurs when user input is used in file paths without validation. Attackers can use `../` sequences to access files outside the intended directory. |
| **File Upload Vulnerabilities** | Occurs when applications allow dangerous files to be uploaded. Attackers may upload web shells to execute code. |
| **Other OWASP Top 10 Vulnerabilities** | Examples include CSRF, command injection, insecure deserialization, and other common web-application security weaknesses. |

---

## 3. Practical Work

Completed multiple authorised training labs:

| Vulnerability | Lab activity | Result |
|---|---|---|
| **SQL Injection** | PortSwigger lab: SQL injection in the WHERE clause. Payload: `' OR 1=1--` | Retrieved hidden products |
| **XSS** | PortSwigger lab: Reflected XSS. Payload: `<script>alert(1)</script>` | JavaScript execution |
| **Authentication** | PortSwigger lab: Username enumeration. Found a valid username and password. | Account access |
| **IDOR** | TryHackMe IDOR room. Changed ID values to access other users' data. | Unauthorized access |
| **Path Traversal** | PortSwigger lab: File path traversal. Payload: `../../../etc/passwd` | Read a system file |

---

## 4. Tools Used

- PortSwigger Web Security Academy
- TryHackMe
- Burp Suite, including Proxy, Repeater, and Intruder
- Browser Developer Tools
- curl

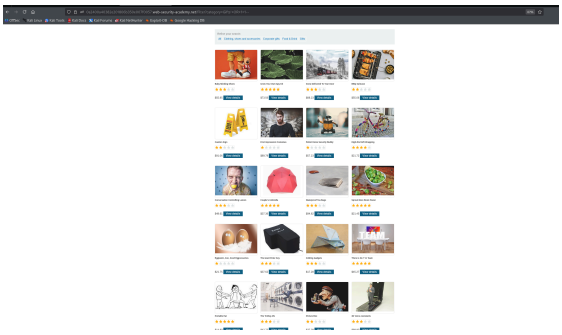
---

# 5. Evidence

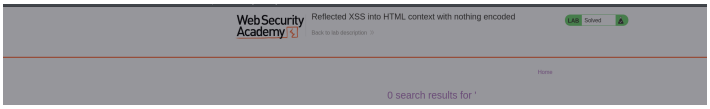
## SQL Injection lab completion



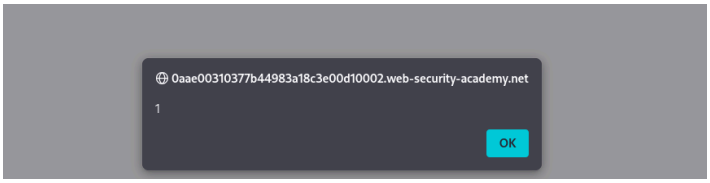

## SQL Injection payload and response



## XSS lab completion



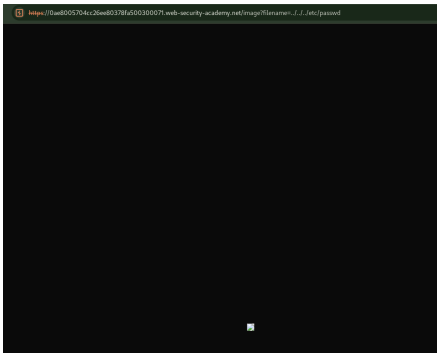

## XSS alert box

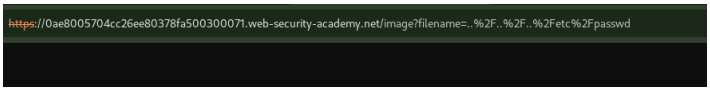



## Path Traversal lab completion

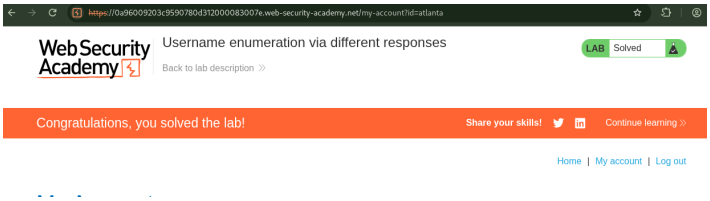



## Path Traversal payload and response

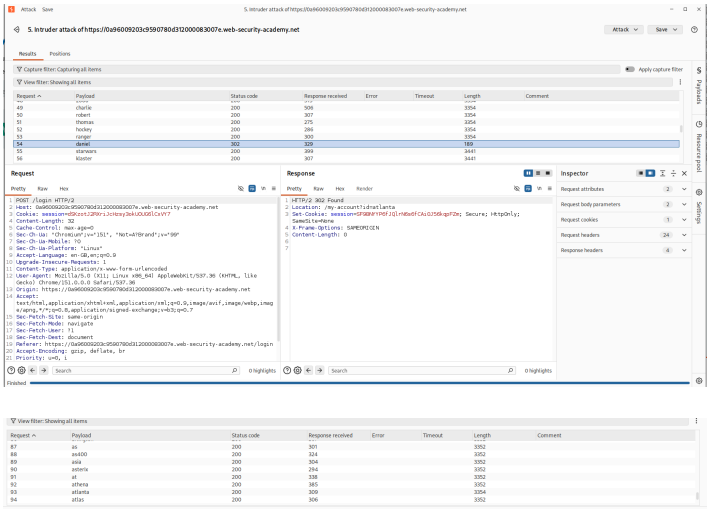




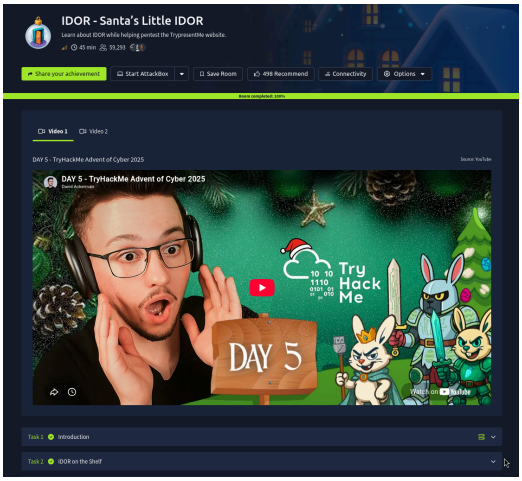
## Authentication lab completion



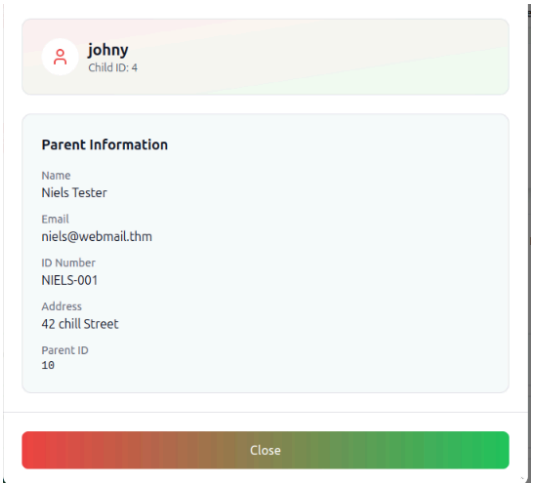

## Burp Intruder results



## IDOR room completion




## IDOR unauthorized access



## 6. Result

The practical work successfully demonstrated the following:

- Exploiting SQL injection to retrieve hidden data
- Executing JavaScript via XSS
- Enumerating valid usernames
- Brute-forcing passwords in an authorised lab environment
- Accessing other users' data via IDOR
- Reading system files via path traversal
- Understanding the root causes and remediation approaches for each vulnerability

---

## 7. Key Learnings

- SQL injection is prevented by using parameterized queries.
- XSS is prevented by applying appropriate output encoding.
- Authentication systems should use generic error messages.
- IDOR is prevented by implementing proper object-level access-control checks.
- Path traversal is prevented through strict input validation and safe file-path handling.
- Access control must be enforced on the server side.
- Every vulnerability has a root cause and a corresponding fix.
- Understanding why a vulnerability occurs is more important than simply demonstrating exploitation.

---

## Conclusion

The practical work demonstrated how common web vulnerabilities can be identified and tested in authorised training environments. It also reinforced the importance of understanding the root cause of each vulnerability and applying the corresponding remediation approach.

---